

BOARD OF INTERMEDIATE EDUCATION, KARACHI

Computer Science XII Lectures

Paper – II

H.S.C. Part II · Science General & Humanities Groups

Option I: Programming Using C · Option II: Programming Using Visual Basic

Both options include DBMS / MS-Access

- C language from structure to functions, arrays and pointers
- DBMS, data models, keys, MS-Access objects and data types
- Visual Basic Option II: controls, events, loops, arrays
- Board programs + solved Model Paper 2026

Original exam-oriented lecture notes · 2026 edition

How Paper–II is set (BIEK 2026)

Paper–II has **two independent options**. Attempt the option you studied. Mixing C and Visual Basic in the same script is not allowed.

Option	Programming	Also asked every year
I	Programming Using C (stdio.h, printf, scanf)	DBMS + MS-Access
II	Programming Using Visual Basic	DBMS + MS-Access

Section	Time	Marks	2026 instruction
A MCQs	20 min	15	All 15 — C MCQs or VB MCQs according to option
B Short	in 2 h 40 min	30	Attempt any TEN parts (3 marks each)
C Detailed	in 2 h 40 min	30	Attempt any THREE questions
Practical	lab	25	Write, compile, run programs; Access tables; viva

EXAM TIP

In C answers, always write **complete programs** with #include, main(), scanf/printf and return 0. Examiners cut marks for missing semicolons, mismatched braces and wrong format specifiers. Indent the body. For DBMS questions, draw tables and mark PK/FK.

Contents — Class XII / Paper–II

Lecture	Title	Option
1	Introduction to C, IDE, program structure	I
2	Tokens, data types, operators, printf/scanf	I
3	if and switch	I
4	Loops, break/continue, board patterns	I
5	Functions	I
6	Arrays, strings, structures	I
7	Pointers and files	I
8	Database fundamentals and data models	I and II
9	MS-Access objects and data types	I and II
10	Visual Basic controls, events, loops	II
11	Practical C program bank	I (lab)
12	MCQ one-liners	Both
13	Solved Model Paper 2026	Both

LECTURE 01**Introduction to Programming and the C Language**

Paper–II Option I · Structure of C

BOARD DEFINITION — Program

A program is a set of instructions written in a programming language that tells the computer how to solve a problem.

BOARD DEFINITION — Programming language

A programming language is a formal language with syntax and semantics used to write programs. Examples: C, C++, Java, Python, Visual Basic.

1.1 Types of errors

Error	When it appears	Example
Syntax	At compile time	Missing semicolon, undeclared variable
Logical	Program runs but result is wrong	Using + instead of * in a formula
Run-time	During execution	Divide by zero, missing file
Linker	After compile, at linking	Forgot to include a library

1.2 History of C (MCQ)

- Developed by **Dennis Ritchie** at Bell Labs in 1972, based on B (Ken Thompson).
- Used to write the UNIX operating system.
- ANSI C / C89 standardised the language; C99 and later added features.
- C is a middle-level, structured, compiled, case-sensitive language.

1.3 IDE**BOARD DEFINITION — IDE**

An Integrated Development Environment is software that combines an editor, compiler, linker and debugger in one place. Examples: Turbo C, Dev-C++, Code::Blocks, Visual Studio.

Typical Turbo C keys (older papers): **Alt+F9** compile, **Ctrl+F9** run, **Alt+F5** user screen.

1.4 Basic structure of a C program (2026 Q.2 i)

BASIC STRUCTURE OF A C PROGRAM

```

#include <stdio.h>    /* preprocessor */

int main() {         /* starting point */

    statements;      /* body */

    return 0;        /* exit status */
}

```

*Figure 1.1 Skeleton every C answer should follow.***PROGRAM / CODE**

```

#include <stdio.h>    /* header for printf / scanf */

int main(void)
{
    /* declarations */
    int a;

    /* input, process, output */
    a = 5;
    printf("%d", a);

    return 0;        /* success */
}

```

*Minimum complete C program.***Parts of the structure**

1. **Preprocessor directives** — lines starting with #, handled before compilation.
2. **#include <stdio.h>** — inserts standard input/output declarations.
3. **#define PI 3.14** — symbolic constant (macro).
4. **Global declarations** — optional; known to every function.
5. **main()** — the function every C program must contain (2026 MCQ).
6. **Body** — statements between { } ending with semicolons.
7. **Comments** — /* multi-line */ and // single line (C99). Ignored by compiler.

1.5 #include and #define (2026 Q.2 ii)**PROGRAM / CODE**

```

#include <stdio.h>    /* standard library header */
#include <math.h>     /* sqrt, pow, abs related */
#define PI 3.14159
#define SQUARE(x) ((x)*(x))

```

Two #include examples and two #define examples.

Header files contain function prototypes. Angle brackets <stdio.h> search the system include path; quotes "my.h" search the project folder first.

LECTURE 02

Tokens, Data Types, Operators and I/O

Paper-II Option I · MCQ + short programs

2.1 Tokens

A token is the smallest unit of a C program: keywords, identifiers, constants, strings, operators, separators.

BOARD DEFINITION — Keyword / reserved word

A reserved word is a word that already has a meaning in C and cannot be used as a variable name. Examples: int, float, char, if, else, for, while, do, switch, case, break, continue, return, void, sizeof. (2026: name any three.)

BOARD DEFINITION — Identifier

A name given to a variable, constant or function. Rules: start with a letter or underscore; then letters, digits, underscore; no spaces; case-sensitive; not a keyword. Valid: Number, NUMBER5, Number_5. Invalid: 5number (starts with digit).

2.2 Data types

Type	Typical size	Format	Range (16/32-bit typical in papers)
char	1 byte	%c	-128 to 127 (signed)
int	2 or 4 bytes	%d	-32768 to 32767 (2-byte)
float	4 bytes	%f	$\sim 10^{-38}$ to 10^{38} , 6 digits
double	8 bytes	%lf	more precision than float
void	none	—	no value (functions)

EXAM TIP

% operator (modulus) works only with **integers**, not float/double. 2026 MCQ.

2.3 Variables and constants

PROGRAM / CODE

```
int age = 17;           /* declaration + initialisation */
const float PI = 3.14; /* constant – cannot change */
char grade = 'A';      /* character constant in single quotes */
char name[] = "Ali";   /* string constant in double quotes */
```

Defining, declaring and initialising.

2.4 Operators

Group	Operators	Notes
Arithmetic	+ - * / %	% only on int; / on int truncates

Group	Operators	Notes
Assignment	= += -= *= /= %=	a += 2 means a = a + 2
Increment	++k (pre), k++ (post)	pre changes then uses; post uses then changes
Decrement	--k, k--	same idea
Relational	< <= > >= == !=	== tests equality; = assigns
Logical	&& !	AND, OR, NOT
Conditional	? :	ternary
Address	&	gives the address of a variable (2026 Q)
Indirection	*	value at a pointer

Precedence (high → low) — 2026 MCQ: highest among + - / %

Parentheses () → ++ -- unary ! → * / % → + - → relational → == != → && → || → ?: → assignment. Among * / % the same level, left-to-right. So between + - / %, / and % are higher than + and -.

Worked expressions (style of 2026 Q.2 x)

If a = 2, b = 5, c = 10:

Expression	Working	Result
200*((a*10)+c)	200*((2*10)+10)=200*(20+10)=200*30	6000
a*b*10/c+a+c	2*5*10/10+2+10 → 100/10+2+10 → 10+2+10	22
a-(c/b)/2+100	2-(10/5)/2+100 → 2-2/2+100 → 2-1+100	101

Integer division: 10/5 = 2, 2/2 = 1. Always show steps.

2.5 printf, scanf, getchar

Function	Header	Use
printf(format, ...)	stdio.h	Output to screen
scanf(format, &vars)	stdio.h	Formatted input; needs address operator &
getchar()	stdio.h	Reads one character (including space)
putchar(ch)	stdio.h	Writes one character
gets / puts	stdio.h	Old string I/O — gets is unsafe; papers still mention it
getch()	conio.h	Reads a key without Enter and without echo (Turbo C)

scanf vs getchar (2026): scanf can read numbers and strings with format specifiers and skips leading whitespace for %d/%f. getchar reads exactly one character, does not need a format, and can read a space or newline. scanf needs &; getchar does not.

Format specifiers and escape sequences

Specifier	Meaning	Escape	Meaning
%d	int	\n	new line

Specifier	Meaning	Escape	Meaning
%f	float	\t	tab
%c	char	\a	alert / beep
%s	string	\\	backslash
%lf	double	\"	quote
%u / %ld	unsigned / long	\0	string terminator (null)

2.6 Convert formulae to C (2026 Q.2 xiii)

Mathematics	C statement
$A = \frac{1}{2} \times \text{base} \times \text{height}$	<code>A = 0.5 * base * height;</code>
$x = 3ab^3 + 3a^2b$	<code>x = 3*a*b*b*b + 3*a*a*b;</code>
$d = b^2 - 4ac $	<code>d = fabs(b*b - 4*a*c); /* need math.h */</code>
$A = \pi r^2$	<code>A = 3.1416 * r * r;</code>

LECTURE 03

Decision Making — if and switch

Paper-II Option I · Section C: switch

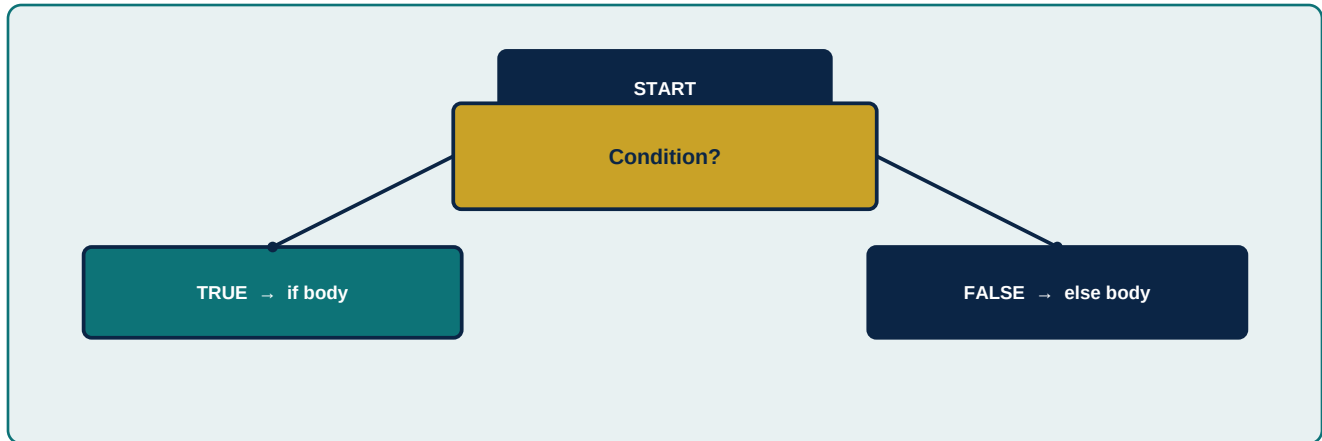


Figure 3.1 Selection structure.

3.1 if, if-else, nested if, else-if ladder**PROGRAM / CODE**

```

#include <stdio.h>
int main(void) {
    int a, b, max;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    if (a > b)
        max = a;
    else
        max = b;
    printf("Greater = %d", max);
    return 0;
}
  
```

*Greater of two numbers — also required as a function in 2026 Q.2 xii.***PROGRAM / CODE**

```

if (marks >= 80)
    printf("A+");
else if (marks >= 70)
    printf("A");
else if (marks >= 60)
    printf("B");
else
    printf("Fail");
  
```

*else-if ladder for a mark sheet.***3.2 switch statement (2026 Section C Q.3)****BOARD DEFINITION — switch**

switch is a multi-way selection statement that compares an integer (or char) expression with several case labels and jumps to the matching block. A break prevents fall-through. default runs when no case matches.

PROGRAM / CODE

```
switch (choice) {  
    case 1:  
        printf("Red colour");  
        break;  
    case 2:  
        printf("Black colour");  
        break;  
    case 3:  
        printf("White colour");  
        break;  
    default:  
        printf("No colour");  
}
```

This is the classic conversion from a nested if-else (past papers).

Rules of switch

1. The switch expression must be int or char (not float).
2. case labels must be constant (not variables).
3. Write break after each case unless you want fall-through.
4. default is optional but recommended.
5. No two cases can have the same label.

In Section C, draw a flowchart: diamond 'choice?' with arrows to case 1, 2, 3 and default.

LECTURE 04

Loops — for, while, do-while, nested

Paper-II Option I · Programs every year

BOARD DEFINITION — Loop / iteration

A loop is a control structure that repeats a block of statements while a condition remains true.

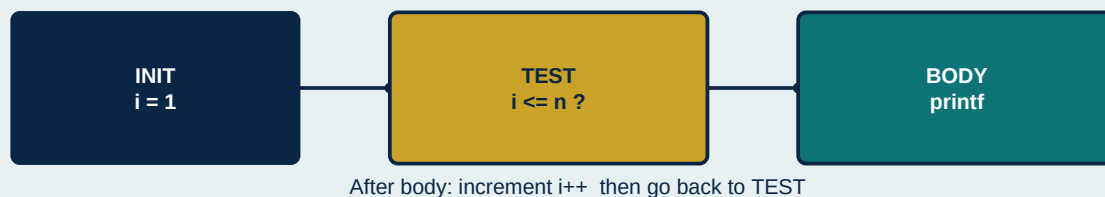


Figure 4.1 for-loop: init → test → body → increment → test ...

4.1 Three loops**PROGRAM / CODE**

```

/* for: used when count is known */
int i;
for (i = 1; i <= 10; i++)
    printf("%d ", i);

/* while: test first – may run zero times */
int n = 1;
while (n <= 10) {
    printf("%d ", n);
    n++;
}

/* do-while: body first – runs at least once */
int m = 1;
do {
    printf("%d ", m);
    m++;
} while (m <= 10);
  
```

Print 1 to 10 with each loop.

for	while	do-while
All three parts in the header	Only condition in header	Condition at the bottom
Best when count is known	Best when count is unknown	Menu / 'try again'
May run 0 times	May run 0 times	Always at least once

4.2 break and continue (2026 Q.2 iv)

break	continue
Leaves the loop (or switch) immediately	Skips the rest of this iteration
Control goes to the statement after the loop	Control goes to the next test / increment
Used to exit early	Used to skip unwanted cases

EXAM TIP

2026 MCQ: 'skip the rest of a loop and carry on from the top' = **continue**, not break.

4.3 Board programs

Multiplication table of an inputted number (2026 Q.2 iii)

PROGRAM / CODE

```
#include <stdio.h>
int main(void) {
    int n, i;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (i = 1; i <= 10; i++)
        printf("%d x %d = %d\n", n, i, n * i);
    return 0;
}
```

Factorial (2026 Q.2 vii) hint 4! = 24

PROGRAM / CODE

```
#include <stdio.h>
int main(void) {
    int n, i;
    long fact = 1;
    printf("Enter n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        fact = fact * i;
    printf("Factorial = %ld", fact);
    return 0;
}
```

Nested loops — two classic patterns (2026 Q.2 xiv)

PROGRAM / CODE

```

/* Pattern A:  1
               12
               123
               1234
               12345          */

int i, j;
for (i = 1; i <= 5; i++) {
    for (j = 1; j <= i; j++)
        printf("%d", j);
    printf("\n");
}

/* Pattern B: number and square
1  1
2  4
3  9
4 16
5 25          */
for (i = 1; i <= 5; i++)
    printf("%d\t%d\n", i, i * i);

```

Even numbers 0–50

PROGRAM / CODE

```

int i;
for (i = 0; i <= 50; i += 2)
    printf("%d ", i);

```

Prime or composite

PROGRAM / CODE

```

int n, i, flag = 0;
scanf("%d", &n);
if (n <= 1) flag = 1;
for (i = 2; i <= n / 2; i++) {
    if (n % i == 0) { flag = 1; break; }
}
if (flag == 0) printf("Prime");
else printf("Composite / not prime");

```

LECTURE 05

Functions

Paper–II Option I · Section C + 2026 greater-number program

BOARD DEFINITION — Function

A function is a named, independent block of code that performs a specific task and can be called from other parts of the program. `main()` is also a function.

5.1 Why functions? (characteristics — 2026 Q.5)

1. **Modularity** — split a large program into small jobs.
2. **Reusability** — write once, call many times.
3. **Easy debugging** — test one block at a time.
4. **Team work** — different programmers write different functions.
5. **Readability** — names describe actions (e.g. factorial).
6. **Returns a value** (or void) to the caller through `return`.

5.2 Prototype, definition, call

PROGRAM / CODE

```
#include <stdio.h>

int greater(int x, int y);    /* prototype / declaration */

int main(void) {
    int a, b, g;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    g = greater(a, b);        /* call – a,b are actual parameters */
    printf("Greater = %d", g);
    return 0;
}

int greater(int x, int y) {   /* definition – x,y are formal parameters */
    if (x > y)
        return x;
    else
        return y;
}
```

2026 Q.2 xii complete program.

Actual parameters	Formal parameters
Values passed at the call	Variables in the function header
a, b in <code>greater(a,b)</code>	x, y in <code>int greater(int x, int y)</code>

5.3 Local vs global (2026 Q in older papers; still useful)

Local (automatic)	Global (external)
Declared inside a function	Declared outside all functions
Known only in that function	Known to every function
Created on entry, destroyed on exit	Lives for the whole program
Prefer these	Use sparingly

5.4 Passing arguments

- **Pass by value** — a copy is sent; changes inside the function do not affect the original.
- **Pass by reference / address** — the address is sent (&x); the function can change the original through a pointer.

LECTURE 06

Arrays, Strings and Structures

Paper-II Option I · Practical + nested programs

BOARD DEFINITION — Array

An array is a collection of elements of the **same data type** stored in contiguous memory, sharing one name and distinguished by an index. The first index is 0.

PROGRAM / CODE

```
int marks[5];           /* declaration, size 5 */
int a[5] = {10, 20, 30, 40, 50}; /* initialisation */
marks[0] = 80;          /* access element */
int i, sum = 0;
for (i = 0; i < 5; i++)
    sum = sum + a[i];
printf("Average = %d", sum / 5);
```

Sort ten numbers ascending (classic Section C)**PROGRAM / CODE**

```
int a[10], i, j, t;
for (i = 0; i < 10; i++)
    scanf("%d", &a[i]);
for (i = 0; i < 9; i++)
    for (j = i + 1; j < 10; j++)
        if (a[i] > a[j]) {
            t = a[i]; a[i] = a[j]; a[j] = t;
        }
for (i = 0; i < 10; i++)
    printf("%d ", a[i]);
```

6.1 Two-dimensional arrays

PROGRAM / CODE

```
int m[3][3], i, j;
for (i = 0; i < 3; i++)
    for (j = 0; j < 3; j++)
        scanf("%d", &m[i][j]);
```

6.2 Strings

A string is a char array ended by the null terminator '\0'. char name[20] = "Karachi";

Function	Header	Job
strlen(s)	string.h	Length without '\0'
strcpy(d,s)	string.h	Copy s into d
strcat(d,s)	string.h	Join s onto the end of d
strcmp(a,b)	string.h	0 if equal; <0 if a<b; >0 if a>b

Reverse a name using strlen

PROGRAM / CODE

```
char s[50];
int i, n;
gets(s);          /* or scanf("%s", s) for one word */
n = strlen(s);
for (i = n - 1; i >= 0; i--)
    putchar(s[i]);
```

6.3 Structures (short)

PROGRAM / CODE

```
struct employee {
    char name[30];
    char desig[20];
    float salary;
};
struct employee e;
printf("Name: "); scanf("%s", e.name);
printf("Pay: ");   scanf("%f", &e.salary);
printf("%s %.2f", e.name, e.salary);
```

LECTURE 07

Pointers (address operator) and a note on files

Paper-II Option I · 2026 asks &

BOARD DEFINITION — Pointer

A pointer is a variable that stores the **memory address** of another variable. The address operator **&** gives an address; the dereference operator ***** gives the value at that address.

PROGRAM / CODE

```
int x = 10;
int *p;      /* p is a pointer to int */
p = &x;      /* p holds the address of x */
printf("Address = %p\n", (void*)p);
printf("Value   = %d\n", *p);    /* 10 */
*p = 25;     /* x becomes 25 */
```

scanf("%d", &n) uses the address operator because scanf must write into n. printf("%d", n) does not need & because it only reads the value.

Double two numbers using pointers (practical list)**PROGRAM / CODE**

```
void double_them(int *a, int *b) {
    *a = *a * 2;
    *b = *b * 2;
}
int main(void) {
    int x = 3, y = 4;
    double_them(&x, &y);
    printf("%d %d", x, y);    /* 6 8 */
    return 0;
}
```

7.1 File handling (short — useful extra)

Text files store characters; binary files store raw bytes. Modes: "r" read, "w" write, "a" append.

PROGRAM / CODE

```
FILE *fp = fopen("notes.txt", "w");
if (fp != NULL) {
    fprintf(fp, "BIEK Computer Science\n");
    fclose(fp);
}
```

LECTURE 08

Database Fundamentals and DBMS

Paper–II both options · 2026 Section B + C

BOARD DEFINITION — Database

A database is an organised collection of related data stored so that it can be retrieved and updated efficiently. Example: student records, library books, bank accounts.

BOARD DEFINITION — DBMS

A Database Management System is software that creates, maintains and controls access to a database. Examples: MS-Access, MySQL, Oracle, SQL Server.

8.1 File system vs DBMS

Flat / file system	DBMS
Data in separate files for each program	Central shared database
High redundancy (same name stored many times)	Controlled redundancy
Inconsistency risk	Integrity constraints keep data consistent
Data depends on the program	Data independence
Weak security	Users, passwords, privileges
Hard to query ad-hoc	SQL / QBE

8.2 Advantages of DBMS (Option II 2026 Q.2 i)

1. Less data redundancy.
2. Data consistency and integrity.
3. Data can be shared by many programs / users.
4. Security and backup.
5. Easy queries, forms and reports.
6. Data independence — change storage without rewriting all programs.

8.3 Components of DBMS (2026 Q.2 xi Option I)

Component	Role
Hardware	Servers, disks, network
Software	The DBMS itself plus OS
Data	Actual stored facts
Procedures	Rules for backup, recovery, use
People / users	DBA, designers, programmers, end users
Data access language	SQL

8.4 Database users

- **DBA (Database Administrator)** — installs DBMS, creates users, backup, performance, security. Responsibilities (2026 VB Q): user accounts, recovery, tuning, enforcing standards.
- **Application programmer** — writes programs / forms that use the database.
- **End user** — enters and reads data through forms (clerk, teacher, cashier).

8.5 Three levels of data abstraction

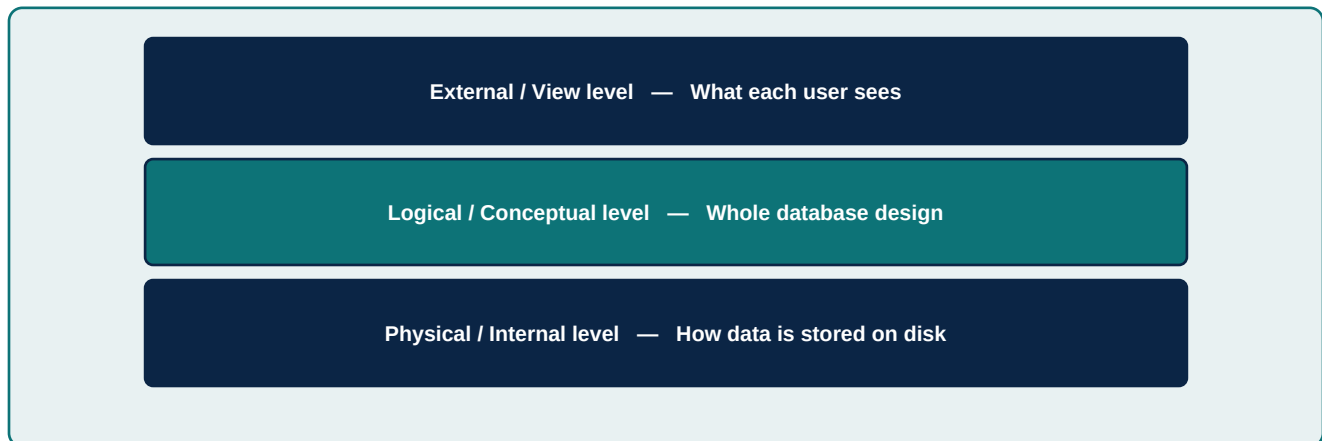


Figure 8.1 External, logical and physical levels.

8.6 Data models (2026 Section C Q.6 both options)

BOARD DEFINITION — Data model

A data model is a set of concepts used to describe the structure of a database — how data is organised and how items relate to each other.

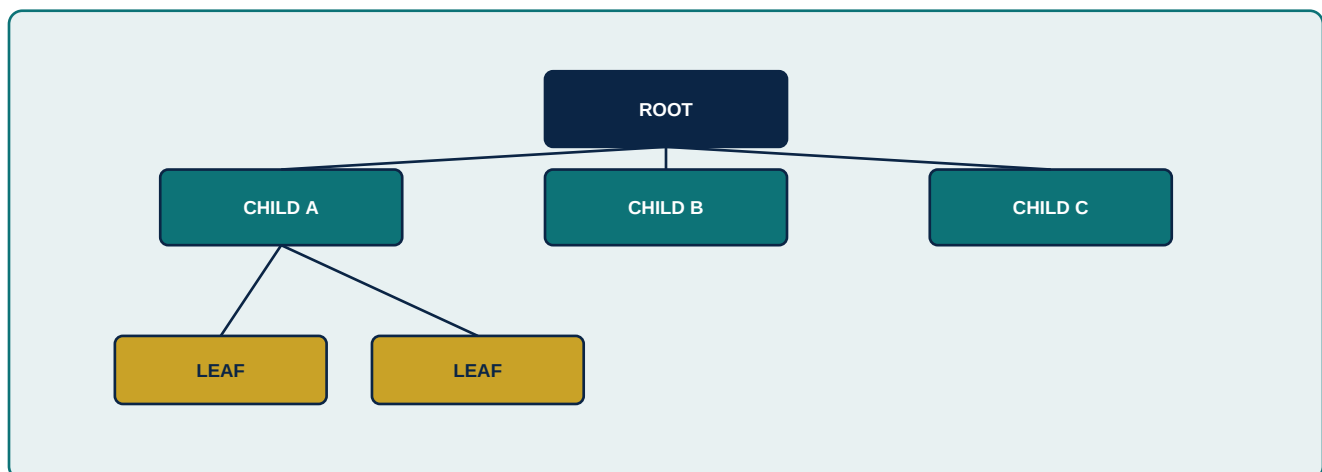


Figure 8.2 Hierarchical model — tree of parent/child records (IMS).

Model	Structure	Plus / minus
Hierarchical	Tree; one parent, many children	Fast for 1:M; hard for M:M
Network	Graph; a child may have many parents (CODASYL)	Flexible; complex pointers
Relational	Tables (relations) with rows and columns; SQL	Simple, used in Access/MySQL

Model	Structure	Plus / minus
E-R model	Entities, attributes, relationships (design drawing)	Used at design time
Object / OR	Objects with methods + tables	Fits OOP applications

2026 MCQ (VB): the model that organises data into a **tree** is Hierarchical. The model that allows SQL is **Relational**.

8.7 Relational terms

Term	Also called	Meaning
Table	Relation	A named 2-D grid of data
Row	Tuple / record	One complete object (one student)
Column	Attribute / field	One property (Name, Marks)
Domain	Data type of a column	Allowed values
Schema	Design of the database	Table names + fields + keys

8.8 Keys

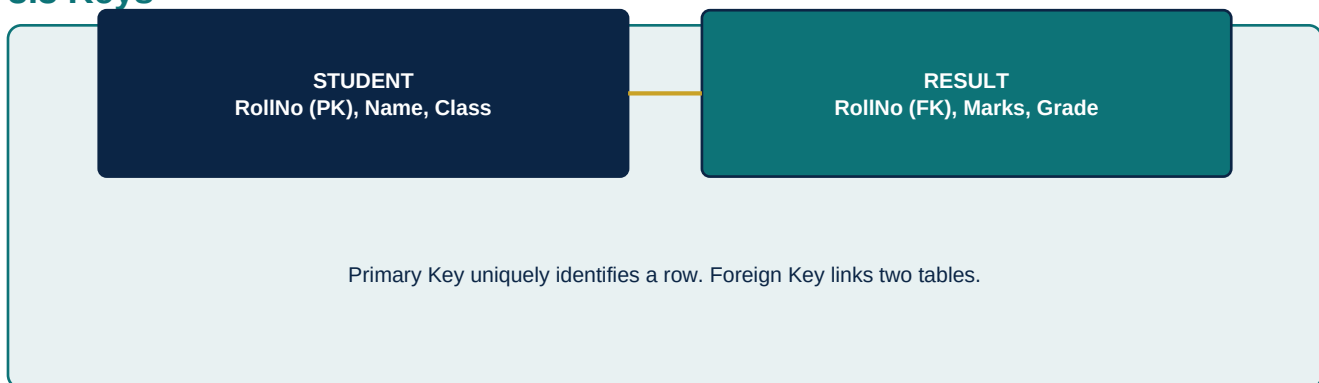


Figure 8.3 Primary key and foreign key.

Key	Definition
Primary key	One or more fields that uniquely identify each record; not null, not duplicate
Foreign key	A field whose values match the primary key of another table — the link
Candidate key	Any field(s) that could be chosen as primary key
Composite key	A primary key made of two or more fields
Super key	Any set of fields that uniquely identifies a row (may have extras)

EXAM TIP

2026 C-option short Q: differentiate Primary key and Foreign key. Write: unique vs linking; in parent vs in child table.

8.9 Relationships

Type	Example
One to one (1:1)	One citizen ↔ one CNIC record

Type	Example
One to many (1:M)	One class ↔ many students (most common)
Many to many (M:M)	Students ↔ courses — needs a third (junction) table

8.10 Integrity and a taste of normalisation

- **Entity integrity** — primary key cannot be NULL.
- **Referential integrity** — foreign key must match an existing PK or be NULL.
- **Domain integrity** — values match the data type / validation rule.
- **1NF** — atomic values, no repeating groups.
- **2NF** — 1NF + no partial dependence on part of a composite PK.
- **3NF** — 2NF + no transitive dependence on non-key fields.

8.11 SQL snapshot

PROGRAM / CODE

```
CREATE TABLE Student (
    RollNo INT PRIMARY KEY,
    SName VARCHAR(40),
    Class VARCHAR(10)
);
INSERT INTO Student VALUES (1, 'Ahmed', 'XII');
SELECT SName, Class FROM Student WHERE RollNo = 1;
UPDATE Student SET Class = 'XI' WHERE RollNo = 1;
DELETE FROM Student WHERE RollNo = 1;
```

Selection chooses rows (WHERE). **Projection** chooses columns (the SELECT list).

LECTURE 09

MS-Access — Objects and Data Types

Paper–II both options · 2026 Q.7 data types

9.1 Objects of MS-Access (2026: define any four / primary object)

Object	Job
Table	Stores data — the primary object (2026 C-option MCQ)
Query	Questions the data; select, update, make-table; SQL behind the scenes
Form	Screen for entering and displaying one record at a time
Report	Printed / PDF output for display (2026 VB MCQ)
Macro	Saved actions (open form, print report)
Module	VBA code

9.2 Data types in MS-Access (write ALL in Section C)

Data type	Stores
Text / Short Text	Names, up to 255 characters
Memo / Long Text	Long remarks
Number	Integers and real numbers (Byte, Integer, Long, Single, Double)
Date/Time	Dates and times
Currency	Money, 4 decimal places, no round-off
AutoNumber	Automatic unique integer — often used as PK
Yes/No	Boolean True/False
OLE Object	Pictures, Word files (older)
Hyperlink	URL or email
Attachment	Files attached to a record (newer Access)
Lookup Wizard	Drop-down from another table or list
Calculated	Value computed from other fields

9.3 Field properties (practical)

- Field Size, Format, Input Mask, Caption.
- Default Value, Validation Rule / Text.
- Required, Indexed, Primary Key button.

9.4 Planning a small database (library example)

1. Purpose: issue books to students.

2. Tables: STUDENT (RollNo PK, Name, Class), BOOK (ISBN PK, Title, Author), ISSUE (IssueID PK, RollNo FK, ISBN FK, Date).
3. 1:M from STUDENT to ISSUE and BOOK to ISSUE.
4. Normalise to 3NF: do not store student name inside ISSUE — look it up.
5. Build tables, relationships (Enforce Referential Integrity), then a query and a report.

LECTURE 10

Visual Basic — Option II

Paper–II Option II · Skip this lecture if you offered C

BOARD DEFINITION — Visual Basic

Visual Basic (VB) is an event-driven, object-based programming language from Microsoft used to build Windows GUI applications. Code runs in response to events such as Click.

10.1 IDE and the form

- Form is the window; its Name, Caption, BackColor, BorderStyle are properties.
- Code window holds event procedures: Private Sub Command1_Click() ... End Sub.
- Toolbox holds ActiveX / intrinsic controls.

10.2 Basic ActiveX / intrinsic controls (2026 Q)

Control	Use	Important properties
Form	Container window	Caption, Name; Cancel button property is of a Button (2026 MCQ)
Label	Display text the user does not type	Caption, Font
Text Box	User types data	Text, MaxLength, PasswordChar, Multiline, Locked
Command Button	Click to run code	Caption, Enabled, Default, Cancel
Check Box	On/off options (many at once)	Value 0/1/2
Option Button	Choose exactly one from a group	Value True/False
Combo Box	Drop-down list + optional typing	Text, List, ListIndex, Style
List Box	List of items	List, ListCount, MultiSelect
Image / Picture Box	Pictures	Picture, Stretch
Timer	Events by interval	Interval (ms), Enabled

Combo Box (2026 Q.2 ii)

A Combo Box combines a text box and a list. Properties: Name, Text, List, ListIndex, Sorted, Style (0 DropDown Combo, 1 Simple, 2 DropDown List). Methods: AddItem, RemoveItem, Clear.

Text Box properties (2026 Q.2 x)

Text, Font, ForeColor, MaxLength, PasswordChar, Multiline, ScrollBars, Alignment, Locked, Enabled, TabIndex, ToolTipText.

10.3 Variables in VB**PROGRAM / CODE**

```
Dim n As Integer      ' explicit type
Dim x As Variant      ' default if type omitted; also: Dim x
Dim name As String
Const PI As Double = 3.1416
```

DIM declares a variable (2026 MCQ). Variant can hold any type and is the default. Passing a copy is **ByVal**; passing the original is **ByRef** (default in older VB).

10.4 Select Case (2026 Q.2 vi)

PROGRAM / CODE

```
Select Case choice
    Case 1
        Label1.Caption = "Red"
    Case 2
        Label1.Caption = "Black"
    Case 3
        Label1.Caption = "White"
    Case Else
        Label1.Caption = "No colour"
End Select
```

10.5 Loops in VB (2026 Section C Q.4)

PROGRAM / CODE

```
Dim i As Integer
For i = 0 To 50 Step 2      ' even numbers 0-50 (2026 Q.2 xiii)
    Print i
Next i

Dim sum As Double, k As Integer, n As Double
sum = 0
For k = 1 To 10             ' sum and average of 10 inputs (Q.2 xiv)
    n = Val(InputBox("Number " & k))
    sum = sum + n
Next k
MsgBox "Sum=" & sum & " Avg=" & sum / 10

Do While i < 10
    i = i + 1
Loop
```

Default Step of For...Next is 1 (2026 MCQ).

10.6 Arrays in VB (2026 Section C Q.3)

PROGRAM / CODE

```
Dim a(1 To 10) As Integer    ' static / fixed array
Dim b() As Integer           ' dynamic
ReDim b(1 To n)
' Control array: several Command buttons sharing one name, Index 0,1,2...
```

Types: one-dimensional, two-dimensional, dynamic (ReDim), control arrays.

10.7 Built-in functions (2026 Q.5)

Function	Job
Val(s)	Text to number
Str(n) / CStr	Number to text
Len(s), Left, Right, Mid	String length and slices
UCase / LCase	Case conversion
Int / Fix / Round	Whole numbers
Sqr, Abs, Sin	Math (need no extra include)
Date, Time, Now	System clock
InputBox / MsgBox	Simple I/O dialogs
IsNumeric	Validation

PROGRAM / CODE

```
Private Sub Command1_Click()
    Dim r As Double, area As Double
    r = Val(Text1.Text)
    area = 3.1416 * r * r
    Label1.Caption = "Area = " & Format(area, "0.00")
End Sub
```

Built-in Val + arithmetic on a Click event.

10.8 VB equivalent of formulae (2026 Q.2 xi)

Math	VB
$X = (ab)^2 / c^3$	$X = (a * b) ^ 2 / c ^ 3$
$Y = (4/3) \pi r^3$	$Y = (4 / 3) * 3.1416 * r ^ 3$
$Y = (a + b)^2 / n^3$	$Y = (a + b) ^ 2 / n ^ 3$
$A = \frac{1}{2} \times \text{base} \times \text{height}$	$A = 0.5 * \text{base} * \text{height}$

10.9 Errors in VB (2026 Q.2 xii)

Type	Example
Syntax	Missing End If, misspelt keyword
Logic	Wrong formula, off-by-one in a loop
Run-time	Divide by zero, type mismatch, file not found

10.10 Full forms (2026 Q.2 ix)

Abbrev.	Full form
ADO	ActiveX Data Objects
MDI	Multiple Document Interface
SQL	Structured Query Language
DBA	Database Administrator

Abbrev.	Full form
OLE	Object Linking and Embedding

LECTURE 11

Practical program bank (C Option I)

Paper-II · Write these in the journal

P1. Simple calculator**PROGRAM / CODE**

```
#include <stdio.h>
int main(void) {
    char op; float a, b;
    printf("Enter a op b: ");
    scanf("%f %c %f", &a, &op, &b);
    switch (op) {
        case '+': printf("%.2f", a + b); break;
        case '-': printf("%.2f", a - b); break;
        case '*': printf("%.2f", a * b); break;
        case '/':
            if (b != 0) printf("%.2f", a / b);
            else printf("Division by zero");
            break;
        default: printf("Unknown operator");
    }
    return 0;
}
```

P2. Four-digit number — print each digit on a new line**PROGRAM / CODE**

```
int n, d1, d2, d3, d4;
scanf("%d", &n);          /* e.g. 2584 */
d1 = n / 1000;
d2 = (n / 100) % 10;
d3 = (n / 10) % 10;
d4 = n % 10;
printf("%d\n%d\n%d\n%d", d1, d2, d3, d4);
```

P3. Factors of n**PROGRAM / CODE**

```
int n, i;
scanf("%d", &n);
for (i = 1; i <= n; i++)
    if (n % i == 0) printf("%d ", i);
```

P4. Search a number in a list**PROGRAM / CODE**

```
int a[10], key, i, found = 0;
for (i = 0; i < 10; i++) scanf("%d", &a[i]);
scanf("%d", &key);
for (i = 0; i < 10; i++)
    if (a[i] == key) { found = 1; break; }
if (found) printf("Found at index %d", i);
else printf("Not found");
```

P5. Add two 3×3 matrices

PROGRAM / CODE

```
int a[3][3], b[3][3], c[3][3], i, j;
for (i = 0; i < 3; i++)
    for (j = 0; j < 3; j++) scanf("%d", &a[i][j]);
for (i = 0; i < 3; i++)
    for (j = 0; j < 3; j++) scanf("%d", &b[i][j]);
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++) {
        c[i][j] = a[i][j] + b[i][j];
        printf("%d ", c[i][j]);
    }
    printf("\n");
}
```

LECTURE 12

MCQ one-liners (C + DBMS + VB)

Paper-II · Last-night revision

1. Every C program must contain main(). Dennis Ritchie founded C.
2. A person who writes application programs is a Programmer.
3. C programs become machine language with a Compiler (not assembler).
4. 5number is an invalid identifier. % works only with int.
5. k++ is post increment. Highest of + - / % is / and % (same level, above + -).
6. continue skips the rest of this iteration. break leaves the loop.
7. Names of variables are identifiers.
8. for is an iteration (loop) structure.
9. Primary key uniquely identifies a row. Table is the primary Access object.
10. DBMS advantage: data is integrated and shared. Report displays/prints data.
11. Columns = attributes. Rows = records/tuples.
12. Hierarchical model is a tree. Relational model uses SQL.
13. Option Button selects one choice; Check Box is independent Yes/No.
14. DIM declares a variable. Default For Step is 1. Cancel button property belongs to Button.
15. A sub program that returns a value is a Function. ByVal = pass a copy.

LECTURE 13

Solved Model Paper 2026 — Paper-II

Paper-II · Option I and Option II

Option I — Programming Using C**Section A (MCQs)**

#	Answer
i	main()
ii	C (Dennis Ritchie)
iii	Programmer
iv	Compiler
v	5number (invalid identifier)
vi	int
vii	k++
viii	/ and % (higher than + -; if only one bubble, pick / or % as printed)
ix	Primary key
x	continue
xi	data is integrated and can be accessed by multiple programs
xii	Table
xiii	identifiers
xiv	Iteration structure
xv	Follow the remaining printed option on the OMR — typically a DBMS/C mix

The official PDF extract of Q.xiv ends at 'Top d...' (top-down). for() is used when we need an **iteration structure**.

Section B — model points (attempt any ten)**Q. i Basic structure of C**

See Lecture 1 code: include, main, declarations, statements, return 0.

Q. ii #include or #define

```
#include <stdio.h>; #include <math.h>; #define PI 3.14; #define MAX 100.
```

Q. iii Multiplication table

Lecture 4 complete for-loop program.

Q. iv break vs continue

Lecture 4 table.

Q. v Reserved words

int, float, return (any three). Cannot be identifiers.

Q. vi Address operator

&x is the address of x; used in scanf("%d", &x) and pointers.

Q. vii Factorial

Lecture 4 program; $4! = 24$.

Q. viii scanf vs getchar

Lecture 2 paragraph.

Q. ix Primary vs Foreign key

Lecture 8 table.

Q. x Expressions $a=2, b=5, c=10$

6000 ; 22 ; 101 (Lecture 2 working).

Q. xi Major components of DBMS

Hardware, software, data, procedures, users, SQL.

Q. xii Greater number using a function

Lecture 5 complete program.

Q. xiii Three formulae in C

Lecture 2 conversion table.

Q. xiv Nested pattern A or B

Lecture 4 nested-loop code.

Q. xv Table, Query, Report

Lecture 9 one-line each.

Section C — pick any three**Q. 3 switch with logical diagram**

Definition, syntax, rules, colour example, flowchart of cases.

Q. 4 Types of operators in C

Arithmetic, assignment, increment, relational, logical, conditional, bitwise (mention), address. One example each.

Q. 5 Characteristics of a function with example

Lecture 5 six points + greater() program.

Q. 6 Data models in DBMS

Hierarchical, network, relational, ER, object — definition + diagram of at least two.

Q. 7 Data types in MS-Access

Write the full table in Lecture 9 with one example value each.

Option II — Visual Basic**Section A**

#	Answer
i	Button (Cancel property)
ii	1 (default Step)
iii	Record
iv	Report
v	Primary Key
vi	Check Box (Yes/No) — if the stem is 0/1; Option Button if 'only one from a group'

#	Answer
vii	Relational (comparison operators)
viii	Pass by value
ix	Return
x	Attributes
xi	Hierarchical model
xii	Option Button
xiii	Function
xiv	Variable (DIM)
xv	Relational

Section B / C

Use Lectures 8–10: four DBMS advantages; Combo Box properties; Dim x As Variant; 1:M relationship diagram; four Access objects; Select Case structure; DBA duties; ActiveX controls; ADO/MDI/SQL/DBA/OLE; Text Box properties; formula conversions; error types; even 0–50 loop; sum and average of ten numbers; PK vs FK. Long: arrays with types; two loops with examples; built-in function program; all data models with diagrams; all Access data types.

Practical checklist — Class XII

1. C: calculator, mark sheet, prime, factorial, table, sort array, reverse string, structure of employee.
2. C: class STUDENT with age and percentage; constructor-style initialisation if your lab uses C++.
3. Pointers: print address and value; double two numbers via pointers.
4. Files: write and read a text file.
5. Access: create table, set PK, change field size, insert/update/delete, SELECT with WHERE.
6. VB (if offered): form with TextBox + Button + Label; even-number loop; area of circle.

End of Class XII / Paper-II lectures. Revise Paper-I theory from the XI book. On exam day, read OR options, write complete C programs, and draw OSI / topology / data-model diagrams in pencil with labels.